Reporte de Penetration Testing

Portada / Metadatos del Engagement

Campo	Valor
Cliente	Laboratorio de Software Seguro — UTN
Título del proyecto	Pentest Web — Reto *Asistencia*
Tipo de evaluación	Black box (reto CTF académico, sin credenciales; endpoint público)
Alcance (scope)	Instancia `https://chl-0db4cda5-4f82-48bc-b073-89c7c232480a-asistencia.softwareseguro.com.ar` y su formulario de verificación `POST /index.php`
Fuera de alcance (out of scope)	Infraestructura de Cloudflare delante de la instancia, otros retos del laboratorio, denegación de servicio
Fechas de ejecución	04/11/2026
Autor / Pentester	Matias Bequis
Versión del documento	1.0
Clasificación	**CONFIDENCIAL**
Distribución	Cátedra de Software Seguro y autor

> **Aviso de confidencialidad:** Este documento contiene información sensible sobre vulnerabilidades. Su divulgación no autorizada puede exponer a la organización a riesgos. Tratar bajo NDA.
>
> **Nota sobre la instancia:** el subdominio `chl-<uuid>-asistencia...` es efímero y se regenera cada vez que se relanza el reto. El UUID consignado aquí corresponde a la instancia usada durante la ejecución; la vulnerabilidad es de la aplicación, no de esa URL puntual.

Control de Versiones

Versión	Fecha	Autor	Cambios
1.0	04/11/2026	Matias Bequis	Versión inicial

---

1. Resumen Ejecutivo

1.1 Contexto y objetivo

Se evaluó el reto *Asistencia* del laboratorio de Software Seguro: una aplicación web con un formulario público de "Verificar Código" (`POST /index.php`), donde se ingresa un código numérico de 3 dígitos (`000` – `999`) y una clave. El objetivo fue determinar si un atacante externo, sin autenticación, puede descubrir **cuántos** y **cuáles** códigos de asistencia son válidos, sin conocerlos de antemano.

1.2 Postura general de seguridad

La aplicación responde al formulario de verificación en un tiempo **sistemáticamente distinto** según el código ingresado sea válido o no: un código válido dispara trabajo adicional del lado del servidor y tarda ~1,0–1,8 s, mientras que uno inválido se responde en ~0,3 s. Esa diferencia, observable desde afuera y sin credenciales, permite reconstruir la lista completa de códigos válidos simplemente midiendo el tiempo de respuesta. Sumado a que el espacio de códigos es minúsculo (solo 1000 combinaciones posibles) y no hay rate limiting efectivo, la postura frente a **enumeración no autenticada no es confiable** y requiere corrección.

1.3 Resumen de hallazgos por severidad

Severidad	Cantidad
🔴 Crítica	0
🟠 Alta	0
🟡 Media	1
🔵 Baja	0
⚬ Informativa	0
Total	**1**

1.4 Riesgos de negocio principales

- **Enumeración completa de códigos válidos** → un atacante externo, sin cuenta, puede descubrir la totalidad de los códigos de asistencia válidos (159 en la instancia evaluada) y conocer el número exacto de asistentes, información que debería ser privada.
- **Habilitación de fraude/suplantación** → si esos códigos operan como prueba de asistencia o como factor de acceso, conocerlos permite reutilizarlos o suplantar a un asistente legítimo; la clave deja de ser un secreto cuando el código que la acompaña es descubrible a voluntad.

1.5 Recomendaciones estratégicas

1. **Corto plazo:** igualar el tiempo de respuesta de la verificación para códigos válidos e inválidos (respuesta en tiempo constante), de modo que el tiempo deje de ser un canal de información.

2. **Corto plazo:** implementar rate limiting / throttling y, opcionalmente, CAPTCHA sobre el endpoint de verificación para encarecer la enumeración masiva.
3. **Mediano plazo:** ampliar el espacio de códigos (tokens largos y aleatorios en lugar de 3 dígitos), ya que 1000 combinaciones son enumerables por fuerza bruta incluso sin el canal de timing.
4. **Corto plazo:** agregar monitoreo/alertas de patrones de enumeración (muchos intentos secuenciales desde un mismo origen) y re-test tras la remediación.

---

2. Detalles Técnicos

2.1 Metodología

Evaluación black box siguiendo la **OWASP Web Security Testing Guide (WSTG)**, con foco en pruebas de enumeración por canal lateral (*Testing for Account Enumeration* / *Timing attacks*). Fases:

1. **Reconocimiento:** identificación del endpoint de verificación y de los campos del formulario (`codigo` , `clave`) inspeccionando el HTML de `index.php` .
2. **Enumeración / formulación de hipótesis:** hipótesis de que el servidor hace más trabajo ante un código válido (lookup en base + verificación) que ante uno inexistente, generando una diferencia de tiempo medible.
3. **Calibración:** medición de una línea base de tiempos sobre una muestra de códigos para estimar el tiempo "típico" de un inválido y fijar un umbral.
4. **Explotación:** barrido completo del espacio `000` – `999` midiendo el tiempo de respuesta de cada código, filtrando por los que superan el umbral y reconfirmándolos con múltiples muestras para descartar ruido de red.

2.2 Alcance técnico detallado

Activo	Tipo	Detalle
`chl-0db4cda5-...-asistencia.softwareseguro.com.ar`	Web App	Formulario público de verificación de códigos de asistencia
`POST /index.php`	Endpoint	Recibe `codigo` (3 dígitos) y `clave` ; responde "Verificar Código". El tiempo de respuesta varía según la validez del `codigo`

2.3 Herramientas utilizadas

`curl` para la inspección del formulario y la calibración manual de tiempos, y un script propio en Python (`contar_asistentes.py` , con la librería `requests`) para el barrido completo, el filtrado por status 200, el umbral adaptativo y la reconfirmación de candidatos.

2.4 Resumen de hallazgos técnicos

ID	Hallazgo	Severidad	CVSS	Estado
VULN-001	Timing side-channel en `/index.php` que permite enumerar sin autenticación todos los códigos de asistencia válidos	🟡 Media	5.3	Abierto

---

3. Hallazgos Detallados

VULN-001 — Timing side-channel en la verificación de códigos (`POST /index.php`)

ID	VULN-001
Severidad	🟡 Media
Activo afectado	`POST /index.php` (parámetro `codigo`)
Categoría	OWASP A07:2021 - Identification and Authentication Failures / CWE-208 (Observable Timing Discrepancy) — enumeración vía CWE-203 (Observable Discrepancy)
Estado	Abierto

3.1 Descripción

El formulario de verificación recibe un `codigo` de 3 dígitos y responde si es válido o no. El servidor **no procesa en tiempo constante**: ante un código válido realiza trabajo adicional (lookup del asistente y verificación asociada) que ante un código inexistente no ocurre, porque corta antes. Esa diferencia de procesamiento se traduce en una diferencia de tiempo de respuesta observable desde el cliente: los códigos válidos tardan de forma consistente mucho más que los inválidos. Como el espacio de códigos es de solo 1000 combinaciones y no hay limitación de tasa efectiva, un atacante puede recorrer `000` – `999` midiendo el tiempo de cada respuesta y clasificar cada código como válido o inválido únicamente por su latencia, sin conocer ninguna clave y sin estar autenticado.

3.2 Impacto

Cualquier atacante externo, sin credenciales, puede: - **Enumerar** la totalidad de los códigos de asistencia válidos del sistema (confidencialidad), obteniendo tanto el **conjunto exacto de códigos** como el **número total de asistentes**. - **Preparar ataques posteriores** sobre esos códigos: si funcionan como prueba de asistencia o como un factor de acceso, conocerlos habilita reutilización, suplantación o fuerza bruta dirigida de la clave sobre códigos ya confirmados como válidos.

En este ejercicio el hallazgo se usó con fin legítimo (contar los asistentes válidos del reto), obteniéndose **159 códigos válidos**. El mismo mecanismo, en un sistema real, expone información que debería mantenerse privada sin ninguna barrera de autenticación.

3.3 Métricas CVSS

> *CVSS v3.1. Se asume `C:L` porque la fuga se limita a la validez/enumeración de códigos (no expone registros personales completos), y `AC:L` porque la separación de tiempos observada es muy amplia (~0,3 s vs. ~1 s+), de modo que una sola medición contra una línea base trivial ya discrimina de forma confiable.*

Métrica	Valor
Puntuación Base	5.3
Severidad	Media
Versión CVSS	3.1
Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Desglose del vector:

Métrica	Sigla	Valor elegido	Justificación
Attack Vector	AV	Network (N)	Explotable remotamente vía HTTP contra el endpoint público
Attack Complexity	AC	Low (L)	La diferencia de tiempo es amplia y consistente (~3×); medir una línea base y comparar es trivial y repetible
Privileges Required	PR	None (N)	El endpoint es público; no requiere autenticación alguna
User Interaction	UI	None (N)	No requiere ninguna acción de una víctima
Scope	S	Unchanged (U)	El impacto queda contenido en la propia aplicación
Confidentiality	C	Low (L)	Expone la validez y el conjunto de códigos (y el total de asistentes), pero no registros personales completos
Integrity	I	None (N)	El ataque solo lee/infirió información; no modifica datos
Availability	A	None (N)	No afecta la disponibilidad del servicio

Cálculo (verificación): - ISS = 1 − [(1−0.22)×(1−0)×(1−0)] = 0.22 - Impacto (Scope *Unchanged*) = 6.42 × 0.22 = 1.4124 - Explotabilidad = 8.22 × 0.85 (AV:N) × 0.77 (AC:L) × 0.85 (PR:N) × 0.85 (UI:N) = 3.887 - Base Score = RoundUp(1.4124 + 3.887) = RoundUp(5.299) = **5.3**

> **Nota de calibración de severidad:** si se considerara que una explotación robusta exige muestreo estadístico para vencer el ruido de red (`AC:H`), el puntaje bajaría a ~3.7 (Baja). Se optó por `AC:L` porque la señal medida fue lo bastante limpia como para clasificar con una sola muestra por código.

> 💡 Calculadora oficial: https://www.first.org/cvss/calculator/3.1

3.4 Prueba de Concepto (PoC) — paso a paso

Precondiciones: - Acceso de red al endpoint público `https://chl-0db4cda5-...-asistencia.softwareseguro.com.ar/index.php` (sin credenciales). - `curl` y Python 3 con la librería `requests`.

Pasos:

1. Identificar los campos del formulario inspeccionando el HTML de la página: `bash curl -s https://chl-0db4cda5-...-asistencia.softwareseguro.com.ar/index.php \ | grep -oiE 'name="[^"]+"' # => name="codigo" name="clave"`

2. Medir el tiempo de respuesta de una muestra de códigos al azar para establecer la línea base de un inválido (la `clave` no afecta el timing; se usa cualquier valor): `bash for c in 000 042 123 250 500 999; do curl -s -o /dev/null -w "$c -> %{time_total}s\n" \ -X POST https://chl-0db4cda5-...-asistencia.softwareseguro.com.ar/index.php \ -d "codigo=$c&clave=gochip" done` Salida típica (ninguno válido en la muestra): todos ~0,17–0,25 s. Línea base ≈ 0,3 s.

3. Barrer el espacio completo `000`–`999` midiendo el tiempo de cada respuesta y marcar como **candidato** todo código cuyo tiempo supere el umbral adaptativo (mediana de la línea base × 3 ≈ 0,925 s). El barrido filtra por status `200` para no confundir errores (p. ej. `503`) con señales de timing: ```http POST /index.php HTTP/1.1 Host: chl-0db4cda5-...-asistencia.softwareseguro.com.ar Content-Type: application/x-www-form-urlencoded

codigo=100&clave=gochip ```

4. Reconfirmar cada candidato con 5 mediciones adicionales y conservarlo solo si su **tiempo mínimo** sigue por encima del umbral. Esto descarta picos de red puntuales (un inválido que por ruido saltó una vez a 0,6 s vuelve a ~0,2 s en la remedición y se descarta).

5. Contar los códigos confirmados: son los asistentes válidos.

Resultado observado: Se observó una separación bimodal clara y sin zona gris entre inválidos y válidos:

	Tiempo
Línea base de inválidos (mediana)	**0,308 s**
Umbral (mediana × 3)	0,925 s
Códigos válidos confirmados	**0,973 s – 1,841 s**

El barrido identificó **159 códigos candidatos** en la fase 1 y la reconfirmación (fase 2, 5 muestras c/u) **confirmó los 159** sin descartar ninguno. Resultado final:

```
Asistentes (códigos válidos confirmados): 159
md5(159) = 140f6969d5213fd0ece03148e62e461e
```

El listado completo de los 159 códigos se adjunta en el **Anexo 5.4**.

Evidencia: Adjuntar: (a) captura de la muestra de calibración mostrando tiempos ~0,2 s para inválidos; (b) salida del script con el umbral calculado, los candidatos y la reconfirmación; (c) resultado final con el conteo y el `md5`. Censurar cualquier dato personal real que no sea necesario para la evidencia.

 Evidencia VULN-001 — separación de tiempos y conteo

3.5 Recomendaciones de Remediación

Acción prioritaria: Hacer que la verificación responda en **tiempo constante**, independientemente de si el código existe: realizar el mismo trabajo (lookup y comparación) en ambas ramas y evitar el *early-return* que ahorra procesamiento cuando el código no existe. Así el tiempo de respuesta deja de correlacionar con la validez del código.

Detalle técnico / ejemplo:

```
import hmac

# ❌ Vulnerable: corta antes y hace trabajo extra solo si el código existe
asistente = db.get_asistente(codigo)        # a veces ni consulta
if not asistente:
    return "código inválido"                 # respuesta rápida
if verificar_clave(asistente, clave):        # hashing caro solo para válidos
    return "ok"
return "código inválido"                     # respuesta lenta

# ✅ Corregido: trabajo equivalente y comparación en tiempo constante
asistente = db.get_asistente(codigo) or ASISTENTE_DUMMY   # siempre consulta
ok = hmac.compare_digest(hash_clave(clave), asistente.clave_hash)
ok = ok and (asistente is not ASISTENTE_DUMMY)
return "ok" if ok else "código inválido"     # mismo costo en ambos caminos
```

Medidas complementarias (defensa en profundidad): - **Rate limiting / throttling** por IP y por sesión sobre `/index.php`, con bloqueo progresivo ante muchos intentos secuenciales; opcionalmente CAPTCHA tras N intentos fallidos. - **Ampliar el espacio de códigos:** 3 dígitos (1000 combinaciones) son enumerables por fuerza bruta aun sin el canal de timing; usar tokens largos y aleatorios no adivinables. - **Monitoreo/alertas** de patrones de enumeración (recorrido secuencial del espacio de códigos desde un mismo origen). - Agregar una pequeña demora aleatoria no sustituye la corrección: el ruido se promedia con suficientes muestras. La respuesta en tiempo constante es la medida de fondo.

Esfuerzo estimado: Bajo **Prioridad sugerida:** Corto plazo

Referencias: - CWE-208: Observable Timing Discrepancy — https://cwe.mitre.org/data/definitions/208.html - CWE-203: Observable Discrepancy — https://cwe.mitre.org/data/definitions/203.html - OWASP Top 10 A07:2021 — Identification and Authentication Failures - OWASP WSTG — Testing for Account Enumeration and Guessable User Account

---

4. Conclusión

El reto *Asistencia* presenta un canal lateral de tiempo (*timing side-channel*) en el formulario de verificación: el servidor tarda sistemáticamente más en responder ante un código válido que ante uno inexistente. Combinado con un espacio de códigos mínimo (1000 combinaciones) y sin limitación de tasa, esto permitió, sin autenticación, enumerar la totalidad de los códigos válidos y determinar que hay **159 asistentes**. Igualando el tiempo de respuesta entre códigos válidos e inválidos (procesamiento en tiempo constante) y agregando rate limiting, el canal desaparece y el riesgo queda mitigado. Se recomienda re-test tras la remediación.

---

5. Anexos

5.1 Glosario

- **Timing side-channel (canal lateral de tiempo):** fuga de información en la que el *tiempo* que tarda un sistema en responder revela algo sobre su estado interno (aquí, si un código existe o no), aunque el contenido de la respuesta sea el mismo.
- **Enumeración:** proceso de descubrir sistemáticamente qué identificadores (códigos, usuarios, etc.) son válidos en un sistema.
- **Respuesta en tiempo constante:** diseño en el que una operación tarda lo mismo independientemente de los datos, para no filtrar información por el tiempo.
- **PoC:** Prueba de Concepto; demostración reproducible de que la vulnerabilidad es explotable.

5.2 Referencias y estándares

- OWASP Top 10 — https://owasp.org/Top10/
- OWASP Web Security Testing Guide (WSTG)
- CVSS v3.1 — https://www.first.org/cvss/
- CWE — https://cwe.mitre.org/

5.3 Metodología de clasificación de severidad

Severidad	Rango CVSS	Criterio de acción
🔴 Crítica	9.0 – 10.0	Remediar de inmediato
🟠 Alta	7.0 – 8.9	Remediar en el corto plazo
🟡 Media	4.0 – 6.9	Planificar remediación
🔵 Baja	0.1 – 3.9	Remediar según recursos
⚬ Informativa	0.0	Buena práctica / mejora

5.4 Anexo — Códigos válidos enumerados (159)

```
100, 105, 107, 115, 121, 129, 133, 134, 140, 160, 161, 163, 168, 172, 173, 174, 177, 183,
189, 194,
205, 207, 211, 218, 219, 221, 227, 236, 244, 247, 253, 255, 266, 274, 277, 281, 284, 287,
292,
302, 305, 306, 307, 308, 309, 317, 319, 332, 342, 343, 346, 347, 349, 362, 367, 371, 373,
374, 397,
405, 407, 447, 454, 457, 463, 464, 468, 477, 480, 488, 493, 494, 497,
506, 513, 516, 519, 520, 525, 529, 530, 533, 535, 543, 550, 551, 552, 554, 567, 568, 572,
575, 579, 589,
604, 610, 611, 613, 616, 618, 623, 637, 651, 653, 658, 675, 678, 682, 683,
704, 719, 725, 729, 735, 736, 742, 746, 749, 753, 766, 771, 775, 785, 787, 793, 798,
803, 813, 819, 820, 824, 831, 839, 844, 846, 847, 854, 859, 865, 880, 883, 890, 896, 898,
903, 910, 911, 917, 923, 938, 950, 956, 958, 967, 979, 982, 990, 993, 997
```

`md5(159) = 140f6969d5213fd0ece03148e62e461e`

Documento generado con la plantilla de reporte de pentest de Matias Bequis — v1.0